

ValorAI: Governed Agentic Intelligence for Real Estate Decision Making

1. Introduction

ValorAI is a governed agentic intelligence platform designed for spatial analysis, valuation modeling, and conversational reasoning in emerging real estate markets. Built as a staging-grade system for the Egyptian property market, it addresses the core vulnerability of conversational applications: the direct coupling of Large Language Models (LLMs) to reasoning, math, and data retrieval tasks, which leads to hallucinations and transactional instability.

ValorAI implements a strict separation of reasoning and computation. All mathematical valuations, spatial containment checks, and comparable retrievals are executed by deterministic backends written in Python and PostgreSQL (PostGIS). The conversational LLM acts strictly as a presentation layer, narrating pre-computed facts and linking them to citation tokens. A governance gate intercepts LLM responses, validating they match composed facts and reverting to a deterministic JSON payload upon failure. This architecture provides deterministic valuation outputs, grounded response generation, and an explainable, hallucination-resistant control plane.

2. Project Objectives

- **Standardize Unstructured Property Markets:** Establish reliable valuation pipelines in opaque markets (like Egypt) that lack centralized MLS databases by normalizing crawled listing data.
 - **Enforce System Governance:** Prevent LLM hallucinations and mathematical errors by offloading computational and spatial reasoning to a deterministic "Truth Layer."
 - **Build Explainable Valuations:** Back property pricing with statistical confidence ratings, spatial comparable properties, and feature impact analysis.
 - **Enable Scenario modeling:** Support multi-tenant, workspace-isolated "what-if" simulations (e.g., estimating value changes from renovations or amenity adjustments).
-

Architectural Philosophy

ValorAI is built around a fundamental principle: language models should not perform valuation calculations, database retrievals, spatial reasoning, or business-critical computations. These responsibilities belong to deterministic services. The language model acts as a controlled narration layer operating on pre-computed and validated outputs.

3. Key Contributions

- **Deterministic Truth Layer:** A modular system of eight analytical tools (Valuation, Explainability, Comparable, Fairness, What-if, Negotiation, Investment, and Market Insight) processing calculations in Python, separating computation from the conversational agent.

- **Goldilocks Hybrid Router:** A density-aware routing service that maps coordinates to H3 spatial grid cells and dynamically selects the optimal pricing model based on geographic data exposure.
- **Explainable Valuation Framework:** A statistical module generating multi-dimensional confidence scores, mapping SHAP values from machine learning log-space to EGP price adjustments, and computing Jaccard-weighted amenity similarity.
- **Spatial Intelligence Platform:** A hierarchical PostGIS database structure that uses recursive SQL queries to resolve parent-child administrative boundaries in under 10ms.
- **Agentic Orchestration Architecture:** A 6-stage runtime pipeline that parses user intent, schedules thread-isolated tool runs, offloads arithmetic calculations, and runs output grounding validation gates.

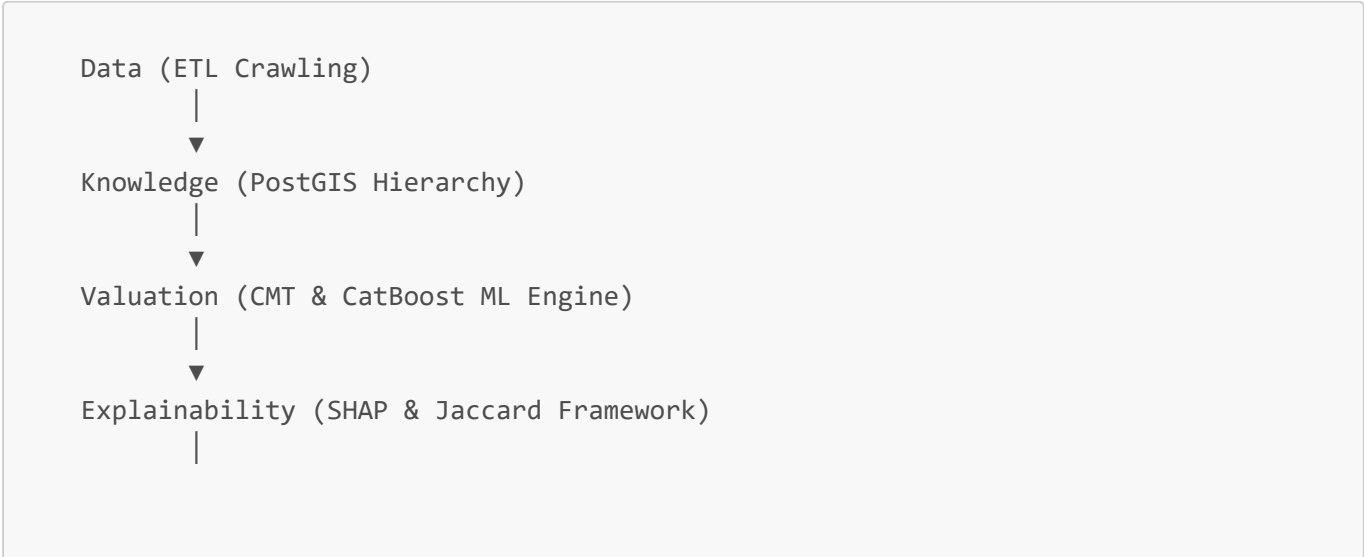
4. System Architecture Overview

The ValorAI architecture enforces a unidirectional flow through six functional layers:

1. **Data Layer:** Handles session-based HTTP scraping of property portal `__NEXT_DATA__` blocks, batch sanitization (filtering listings with price ≤ 100 EGP or size outside $5 - 10,000$ sqm, enforcing coordinate bounds), and address geocoding caches.
2. **Spatial Intelligence Layer:** Houses PostgreSQL/PostGIS geometries. Uses spatial indices (GiST) and recursive area walks to locate listings and verify coordinate containment.
3. **Valuation Layer:** Houses the rule-based Comparable Market Technique (CMT) engine and the CatBoost machine learning (ML) models, coordinated by the Goldilocks Router.
4. **Explainability Layer:** Evaluates multi-dimensional confidence ratings, feature-level SHAP impact adjustments, and weighted amenity overlaps.
5. **Agentic Orchestration Layer:** Standardizes natural language processing. Maps user requests to tool execution graphs, runs thread-isolated database operations, offloads comparisons, and validates outputs.
6. **Presentation Layer:** The mobile Flutter client and authenticated API gateways routing user requests via Bearer JWT and Server-Sent Events (SSE).

5. Engineering Evolution Journey

The project evolved through five distinct architectural phases, mapping from raw data to governed agentic intelligence:



▼ Agentic Intelligence (Orchestration Control Plane)

1. **Data Phase:** Ingested raw property portals SSR HTML pages using session-based crawlers. Engineered in-memory cleaning filters to prune duplicate posts, enforce bounding boxes, and handle bilingual studio classifications.
2. **Knowledge Phase:** Upgraded database from flat records to a 4-level parent-child administrative hierarchy (Governorate → City → District → Compound/Neighborhood). Integrated PostGIS spatial polygon geometries (`ST_Covers`) to prevent coordinate-to-neighborhood errors.
3. **Valuation Phase:** Developed dual pricing systems: a statistical CMT engine sweeping five spatial search tiers, and CatBoost ML regressors trained on H3 spatial grid cells to interpolate prices.
4. **Explainability Phase:** Designed statistical frameworks to generate trust. Integrated SHAP feature drivers, unified confidence metrics, and category-weighted Jaccard indexes for amenity matching.
5. **Agentic Phase:** Built a 6-stage orchestrator to manage multi-turn dialogues. Implemented narration contracts and grounding gates to enforce a deterministic fallback, preventing LLMs from calculating or hallucinating values.

6. Spatial Intelligence Platform

The spatial intelligence platform resolves raw coordinate inputs into authoritative geographic areas and retrieves local comparable listings. The database is built on PostgreSQL 15 and PostGIS:

- **Administrative Hierarchy:** Area boundaries are mapped using a 4-level parent-child hierarchy (`areas` table). When coordinates are received, a recursive Common Table Expression (CTE) walks up parent references using `ST_Covers` to resolve the full administrative path (e.g., Compound → District → City → Governorate) in under 10ms.
- **Geospatial Indexing:** The schema uses GiST (Generalized Search Tree) indexes on spatial polygon shapes (`geom`) and point centroids (`center_geom` geog) to optimize radius searches (`ST_DWithin`).
- **GIN Indexes:** Generalized Inverted Indexes are applied to the listing amenities JSONB columns (`normalized_amenities`) to bypass string parsing and speed up key-value containment filters.

7. Valuation Architecture

The valuation engine splits execution between a rule-based statistical model and a machine learning regressor.

Comparable Market Technique (CMT) Engine

The CMT engine replicates the manual appraisal process. It retrieves comparable properties across five expanding search tiers (Compound → District → Adjacent Districts → City → Governorate) until a target sample size ($N \geq 40$) is reached.

- **Outlier Removal:** The retrieved sample is pruned using the Median Absolute Deviation (MAD). For samples ≥ 10 , listings with a Modified Z-score $|MZ_i| > 3.5$ based on price per square meter are discarded. A Zero-MAD exception defaults to a $\pm 5\%$ median price band.

- **Similarity Weighting:** The remaining comps are weighted based on spatial distance decay, size similarity, listing recency decay, bathroom matching, same leaf-area rewards, property type compatibility, and feature similarities.
- **Valuation Quantiles:** Valuations are resolved via a weighted quantile picker. Expected Fair Price is resolved at the P50 median, while price range boundaries are set at P20 (lower bound) and P80 (upper bound).

Machine Learning (ML) Engine

The ML engine uses CatBoost regressor models (`residential_rent.cbm` and `residential_sale.cbm`) to predict property prices in sparse data regions.

- **Inference Pipeline:** Inputs include coordinates, property size, room counts, furnishing status, and Uber H3 hexagonal grid cells (resolutions 8 and 9).
- **Mathematical Inversion:** The models are trained on log-transformed targets ($y = \ln(\text{price} + 1)$) to stabilize variance across property tiers. Predictions are inverted back to EGP ($e^{\hat{y}} - 1$) at inference time.

Architectural Rationale & Trade-offs

- **CMT Engine:** Provides high audit transparency and evidence-backed appraisals, but fails in cold-start regions where listing density is low.
- **ML Engine:** Interpolates prices across sparse coordinate grids and scales efficiently, but behaves as a "black box" that lacks comparable physical evidence.

8. Hybrid Routing Research

The routing layer evolved through three versions to combine CMT's transparency with ML's coverage:

- **Hybrid V1 (Direct Fallback - Failed):** The router ran CMT first. If CMT failed to find comparables, the system triggered the CatBoost model. In sparse areas, the spatial geofencing loop frequently matched 1-2 highly stale, distant listings in the outer search tiers. The engine calculated valuations on this sparse data, producing highly distorted estimates instead of falling back to the ML model.
- **Hybrid V2 (Density Threshold - Failed):** The router executed CMT if the comparable count was ≥ 10 , falling back to ML otherwise. In high-density districts (e.g., Madinaty with > 700 comps), calculating similarities and sorting rows in Python caused CPU latency spikes of up to 300ms. In addition, CMT valuations in dense areas were overly sensitive to listing anomalies.
- **Hybrid V3 (Goldilocks Router - Succeeded):** The active implementation uses an exposure registry (`exposure_registry.json`) to check historical data density by H3 index resolution 9 cells. The router evaluates density rules before executing valuations:
 - **Goldilocks Zone** ($11 \leq N \leq 50$): Routes to CMT, ensuring highly accurate, evidence-backed appraisals without CPU processing bottlenecks.
 - **Unseen with Sufficient Comps** ($N \geq 10$): Routes to CMT in unseen areas to establish a verifiable baseline.
 - **ML Default** ($N < 11$ or $N > 50$): Routes to CatBoost ML, which processes predictions in under 15ms in dense zones and interpolates regional trends in sparse zones.
- **Shadow Execution & Telemetry:** FastAPI `BackgroundTasks` run the alternate model in the background (ML if CMT is active, or CMT if ML is active) and log results to `shadow_logs` to monitor model drift.

9. Explainability Framework

The explainability framework translates backend mathematical models into human-readable narratives:

- **Multi-Dimensional Confidence Scoring:** Generates a unified confidence rating ($[0.0, 1.0]$) combining sample size, tier levels, MAD filtering stability, price dispersion ($\frac{P_{80}-P_{20}}{P_{50}}$), average distance decay, and listing recency. Ambiguous geocoding or fallback distances exceeding 2,500m cap the score at 0.50.
- **SHAP Explanations:** For ML valuations, log-space feature impacts are converted to EGP adjustments ($\text{Price} \cdot (e^{\text{SHAP}_i} - 1)$) to identify the top positive and negative value drivers.
- **Jaccard-Weighted Amenities:** Scores comparable similarity using a category-weighted Jaccard index, ensuring high-value amenities (like parking or private pools) impact similarity scores more than secondary features.
- **Narrative Traces:** Compiles coordinates resolution, database search tiers, filtered anomalies, and amenity details into a structured trace.

10. Agentic Orchestration

The agentic orchestrator manages natural language interactions through six sequential runtime stages:

1. **Intent Engine:** Uses rule-based keyword matching to classify messages into a 10-intent taxonomy (e.g., **VALUATION**, **NEGOTIATION**, **WHAT_IF**), preventing prompt injections and latency.
2. **Tool Planner:** Maps intents to execution graphs. Parallel queries (like side-by-side property comparisons) generate concurrent tool calls.
3. **Tool Executor:** Dispatches queries to internal tools using isolated database sessions, preventing database access leaks and isolating runtime timeouts.
4. **Response Composer:** Normalizes data payloads. Under the **Arithmetic Offloading Rule**, the composer calculates all averages, ranges, and deltas in native Python before prompt assembly, and truncates listings arrays to fit context limits.
5. **Memory Layer:** Manages conversation history and scenario branches. Database lookups are strictly filtered by the user's JWT-validated **workspace_id** to maintain multi-tenant security.
6. **Narration Layer:** Streams the response using the LLM. Instructs the model to act as a narrator of the pre-computed facts. If generated text violates grounding validation rules, the Narration Admission Gate overrides the output.

11. Governance and Grounding

To guarantee computational accuracy, ValorAI enforces strict boundary rules:

- **Separation of Concerns:** The LLM has zero database access and is blocked from performing reasoning, coordinates matching, or calculations. It operates only on a bounded context payload.
- **Narration Contracts:** System prompts instruct the LLM to restate facts and structured citation tokens exactly, and forbid it from generating new calculations, valuations, or recommendations.
- **Grounding Validation & Fallback:** The Narration Admission Gate monitors generated text. All numerical values must match numbers in the upstream context, and citation tokens must match active IDs:

[citation:<exact-received-id>]

If the LLM creates ungrounded values or invalid citations, the system suppresses the generated text and triggers a **Deterministic Fallback**, returning the raw JSON payload (**DETERMINISTIC_FALLBACK**) to the client.

12. Current Implementation Status

- **Core Backend Database:** PostgreSQL 15 migrations (**000-007**), spatial schemas, recursive views, and GiST/GIN indexes are implemented and operational.
- **Valuation Engines:** The multi-tier CMT engine, CatBoost ML inference pipelines, and the Goldilocks Router are fully functional.
- **Orchestration Runtime:** Intent classification, tool planners, responders, and memory layers pass automated validation checks.
- **Staging Validation:** Core systems build successfully and pass integration tests within staging Docker environments.

13. Limitations and Current Gaps

- **Authentication Gaps:** Firebase token exchange is operational on the backend, but client integration is incomplete. Missing Android and iOS service credential files block physical device authentication.
- **Role-Based Access Control (RBAC):** Memory tables enforce workspace boundaries, but the system lacks fine-grained user roles (e.g., administrator, broker, client).
- **Secrets Management:** The platform reads passwords and API keys from local environment files, lacking a production-grade secret management and rotation vault.
- **LLM Narration Default-Off:** The Gemini LLM adapter is disabled by default in staging until API keys and data-governance credentials are set.

14. Future Roadmap

- **Production Hardening:** Integrate a production key manager for credential storage, rotate secrets, and complete frontend JWT validation tests.
- **Expanded Spatial Coverage:** Extend PostGIS polygon boundary registries and H3 spatial indexes to additional metropolitan markets.
- **Advanced Valuation Models:** Add temporal trend indices to CMT weights to adjust for inflation, and train commercial-specific regressors.
- **Advanced Agent Capabilities:** Implement multi-user portfolio tracking and collaborative workspace sharing features.
- **Conversational Deployment:** Enable the LLM narrator in production, protected by the Narration Admission Gate and the deterministic JSON fallback.

Performance & Evaluation

Performance benchmarking and model evaluation are ongoing. Under staging validation, PostGIS parent-child spatial hierarchy queries consistently execute in under 10ms, address resolution geocoding cache lookups resolve in under 5ms, and the CatBoost ML engine processes inference in under 15ms. Future releases will include comprehensive valuation accuracy metrics (such as MAPE, MAE, and R^2), routing effectiveness analysis, spatial query scale benchmarks, and end-to-end latency measurements.

15. Conclusion

ValorAI demonstrates how mathematical, spatial, and machine learning models can be integrated under a deterministic control plane. By decoupling computation from language generation, the system ensures that property valuations and comparative analyses remain statistically sound. The mobile client is a presentation channel; the core engineering contribution lies in the PostGIS hierarchies, the density-aware routing, the explainability metrics, and the orchestration plane that enforces data integrity. This design provides a reliable blueprint for building secure, explainable, and architecturally governed AI systems in complex domains.